National University of Computer & Emerging Sciences

Lecture 10

Interprocess Communication

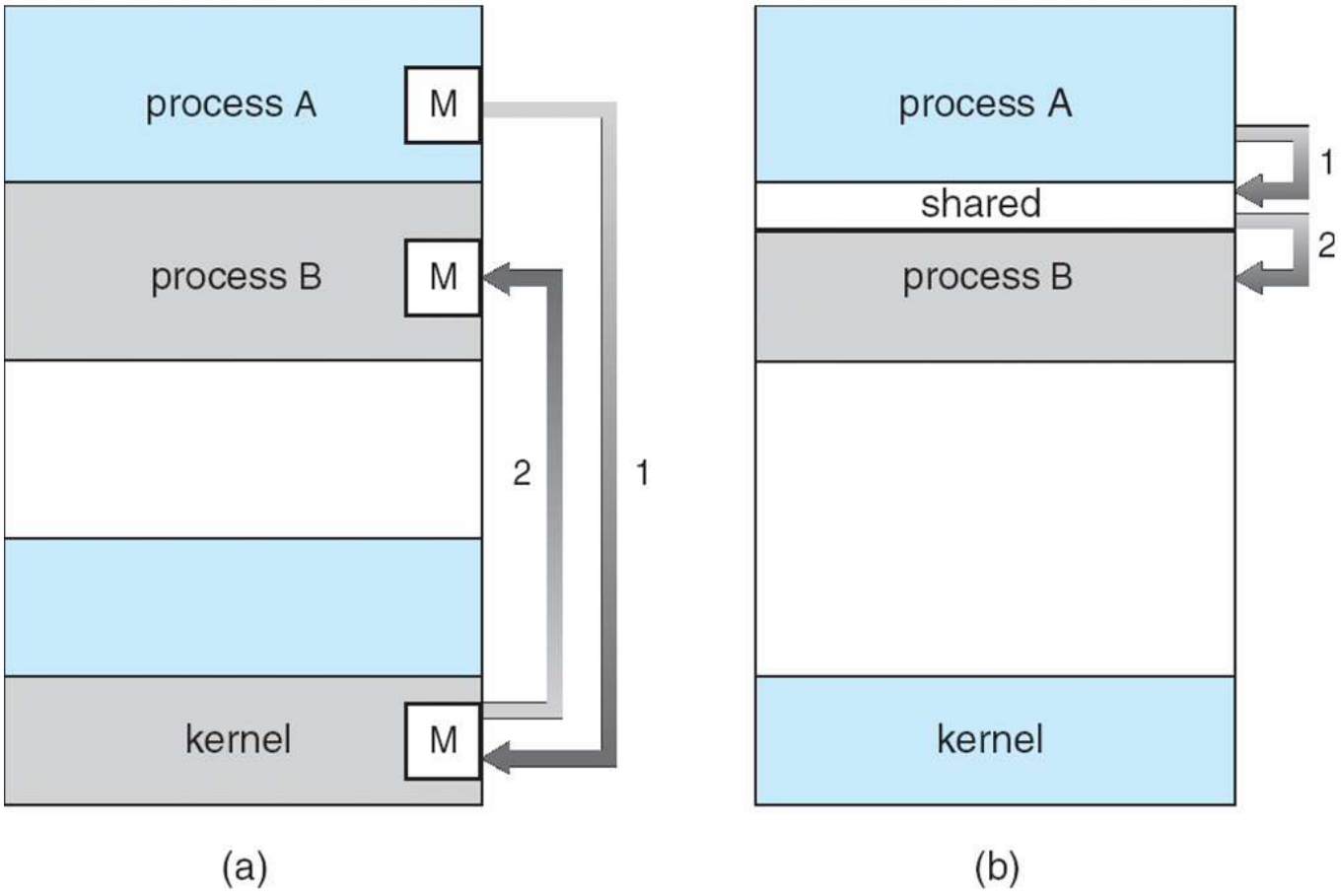
Interprocess Communication

- A process has access to the memory which constitutes its own address space.
- When a child process is created, the only way to communicate between a parent and a child process is:
  - The parent receives the exit status of the child
- So far, we've discussed communication mechanisms only during process creation/termination
- Processes may need to communicate during their life time.

Interprocess Communication

- Processes may be **independent** or **cooperating**
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Dangers of process cooperation
  - Data corruption, deadlocks, increased complexity
  - Requires processes to synchronize their processing
- Two models of IPC
  - Shared memory
  - Message passing

Communications Models



IPC Mechanisms

- Mechanisms used for communication and synchronization
  - Message Passing
    - message passing interfaces, mailboxes and message queues
    - sockets, pipes
  - *Shared Memory*: Non-message passing systems
  - *Event Notification* - UNIX *signals*
  - *Synchronization* – primitives such as semaphores to higher level mechanisms such as monitors

- We will defer a detailed discussion of synchronization mechanisms and concurrency until a later class
- Here we want to focus on some common (and fundamental) IPC mechanisms

Interprocess Communication – Message Passing

- Mechanism for processes to communicate and to synchronize their actions
- Message system – processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*) – message size fixed or variable
  - **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
  - establish a *communication link* between them
  - exchange messages via send/receive
- Implementation of communication link
  - physical (e.g., shared memory, hardware bus)
  - logical (e.g., logical properties)

Implementation Questions

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?

Message Passing Systems

- Exchange messages over a communication link
- Methods for implementing the communication link and primitives (*send*/*receive*):
  1. Direct or Indirect communications (*Naming*)
  2. Symmetric or Asymmetric communications (blocking versus non-blocking)
  3. Buffering
  4. Send-by-copy or send-by-reference
  5. fixed or variable sized messages

Direct Communication - Internet and Sockets

- Processes must name each other *explicitly*:
  - *send (**P**, message)* – send to process P
  - *receive(**Q**, message)* – receive from Q

- Properties of communication link
  - Links established automatically between pairs
  - processes must know each others ID
  - Exactly one link per pair of communicating processes
- Disadvantage: a process must know the name or ID of the process(es) it wishes to communicate with

Indirect Communication

- Messages are sent to or received from *mailboxes* (also referred to as *ports*).
  - Each mailbox has a unique id.
  - Processes can communicate only if they share a mailbox.
- Primitives:
  - *send(A, message)* – send a message to mailbox A
  - *receive(A, message)* – receive a message from mailbox A
- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with more than 2 processes.
  - Each pair of processes may share several communication links.

Indirect Communication-Ownership

- process owns (i.e. mailbox is implemented in user space):
  - only the owner may receive messages through this mailbox.
  - Other processes may only send.
  - When process terminates any "owned" mailboxes are destroyed.
- kernel owns
  - then mechanisms provided to create, delete, send and receive through mailboxes.
  - Process that creates mailbox owns it (and so may receive through it)
  - but may transfer ownership to another process.

Indirect Communication

- Mailbox sharing:
  - P_1, P_2, and P_3 share mailbox A.
  - P_1, sends; P_2 and P_3 receive.
  - Who gets the message?
- Solutions
  - Allow a link to be associated with at most two processes.
  - OR Allow only one process at a time to execute a receive operation.
  - OR Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was

Synchronization

- Message passing may be either *blocking* or *non-blocking*.
  - blocking send:
    - sender blocked until message received by mailbox or process
  - nonblocking send:
    - sender resumes operation immediately after sending
  - blocking receive:
    - receiver blocks until a message is available
  - nonblocking receive:
    - receiver returns immediately with either a valid or null message.

Buffering

- All messaging system require framework to temporarily buffer messages.
- These queues are implemented in one of three ways:
  1. Zero capacity

     No messages may be queued within the link, requires sender to block until receiver retrieves message.
  2. Bounded capacity

     Link has finite number of message buffers. If no buffers are available then sender must block until one is freed up.
  3. Unbounded capacity

     Link has unlimited buffer space, consequently send never needs to block.